Implementation Plan

AI-Powered Anxiety & Stress Management Mobile Application with Intelligent Risk Prediction

Flutter | Node.js + Express | PostgreSQL | Python ML | LLM Assistant

Scope: engineering delivery only (UI/UX design excluded) | Timeline: 16 weeks

Important: This is a wellness-support application, not a medical diagnostic tool. It estimates *stress risk* from self-reported and optional wearable data. Models are not clinically validated, and the supplied dataset is synthetic.

1. Overview and Assumptions

- Team of 2-3 developers (a solo developer should extend the timeline to about 24 weeks and drop stretch items).
- UI/UX designs are produced separately in Figma and handed over per screen; engineering consumes them.
- Synthetic dataset and training scripts (generate_dataset.py, train_model.py) are already available.
- Primary demo platform is Android; iOS (HealthKit) is optional because it needs a Mac, a developer account and a physical iPhone.

2. Scope Tiers

Tier	Included features
MVP (must ship)	Authentication, profile setup, mood check-in, daily stress assessment, manual sleep log, habits, dashboard, next-day risk prediction with explanation, recommendations, basic analytics, privacy center (export/delete), emergency screen
Core extension	Context-aware chatbot with safety layer, notification center, activity tracking
Stretch	Wearable sync (Health Connect / HealthKit), dark mode, 90-day analytics and correlations, multi-region emergency resources

If time runs short, cut from the bottom of this list upward.

3. Technology Stack

Layer	Choice
Mobile	Flutter, Riverpod (state), go_router, Dio, fl_chart, flutter_secure_storage, Drift/SQLite (offline queue)
Backend API	Node.js + TypeScript, Express, Zod validation, Prisma or node-postgres, JWT access + refresh tokens, argon2/bcrypt, helmet, express-rate-limit, pino logging, node-cron or BullMQ for scheduled jobs
ML service	FastAPI, scikit-learn, XGBoost, joblib model loading, Pydantic strict schemas
Database	PostgreSQL (19 tables from the specification), SQL migrations, indexes on (user_id, date)
Infrastructure	Docker Compose (local), GitHub Actions CI, deployment on Render / Railway / Fly.io
LLM	One provider API, called only from the backend; keys stored in environment variables

4. System Architecture

Flow	Description
Mobile to API	Flutter app talks to the Node.js REST API over HTTPS; no secrets or LLM keys inside the app.
API to database	All reads and writes are scoped by the authenticated user id; every query checks ownership.
API to ML service	After each check-in (and nightly), Node builds the feature vector and calls the Python /predict endpoint; the result is stored in predictions and prediction_factors.
API to chatbot	Context builder collects minimal summaries, crisis detector runs first, then the LLM, then output validation.

5. Timeline Overview (16 Weeks)

Phase	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0 Setup																
1 Backend core																
2 Flutter core																
3 ML service																
4 Prediction integration																
5 Chatbot																
6 Wearables (stretch)																
7 Privacy and security																
8 Testing and evaluation																
9 Deploy and document																

Orange bars mark stretch work. Phases 1, 2 and 3 run in parallel after setup.

6. Phase-by-Phase Plan

Phase 0 - Setup (Week 1)

- Create monorepo with /app (Flutter), /api (Node), /ml (FastAPI).
- Docker Compose for Postgres, API and ML service; .env.example files.
- SQL migrations for all 19 tables; seed script that imports the synthetic CSVs.
- OpenAPI contract for every endpoint; CI running lint and tests.

Done when: docker compose up starts every service and CI is green.

Phase 1 - Backend core (Weeks 2-4)

- Auth: register, login, refresh, logout, forgot/reset password, email verification.
- Users, preferences and privacy-settings endpoints.
- CRUD for mood, stress assessments, sleep, habits and logs, activity, notifications, emergency contacts.
- Zod validation on every body/query; ownership checks; rate limiting; audit-log middleware.

Done when: All endpoints pass supertest suites and no endpoint can return another user's data.

Phase 2 - Flutter core (Weeks 3-7, parallel)

- Clean-architecture skeleton (presentation / domain / data) per feature; routing and theming.
- Authentication, onboarding and profile setup flows.
- Mood, stress assessment, sleep and habit features wired to the API.
- Dashboard, loading/empty/error/offline states, Drift offline queue with client-generated UUIDs.

Done when: A user can register, check in daily and see the dashboard against the real API.

Phase 3 - ML service (Weeks 4-7, parallel)

- FastAPI endpoints /predict, /health and /model-info.
- Feature builder that creates rolling 3-day and 7-day features from database rows.
- Strict Pydantic input schema with ranges; model versioning; configurable risk thresholds.
- Explanation output (grouped factor impacts) plus confidence.

Done when: Same input gives the same output as the training script; invalid input returns 422.

Phase 4 - Prediction integration (Weeks 7-8)

- Node triggers prediction after each check-in and with a nightly job.
- Persist predictions and factors; store the actual next-day outcome for later evaluation.
- Rule-based recommendation engine driven by top risk-increasing factors.
- Flutter prediction, explanation and recommendation screens; cold-start 'early estimate' mode.

Done when: A new check-in updates tomorrow's risk within seconds.

Phase 5 - Chatbot (Weeks 9-10)

- Context builder sends only summaries (for example 'average sleep 5.8 h, stress trending up').
- Crisis-detection layer runs before the LLM and routes to support options.
- LLM call from backend only; output validation blocks diagnostic or therapist-style claims.
- Chat history, suggested prompts, recommendation cards, deletion of chat data.

Done when: Crisis messages never reach the normal chat flow; the bot never claims to be a clinician.

Phase 6 - Wearables, stretch (Weeks 10-12)

- Flutter 'health' package, permission request flow, device status screens.
- Idempotent /wearables/sync upsert by (device, date).
- Imported sleep and activity override manual entries only when present.
- Full no-device fallback.

Done when: Data syncs on a real Android device and the app works fully without a wearable.

Phase 7 - Privacy and security (Weeks 12-13)

- JSON data export; full deletion cascade for account and per-category data.
- Audit logging of sensitive actions; secure token storage on device.
- HTTPS, CORS allow-list, helmet, rate limits, secrets in environment variables.
- OWASP API Top 10 checklist and dependency scanning.

Done when: Deleting an account removes every row belonging to that user.

Phase 8 - Testing and evaluation (Weeks 13-15)

- Unit tests (Dart, TypeScript, Python), API tests, widget and integration tests.
- ML report: accuracy, precision, recall, F1, ROC-AUC, confusion matrix, baseline comparison.
- Security tests (authorization bypass, injection, rate limit); usability test with 5-10 people.
- Collect consented real check-ins to validate or retrain.

Done when: Every flow in the specification has been walked end to end and defects are triaged.

Phase 9 - Deploy and document (Weeks 15-16)

- Production deployment with managed Postgres and backups.
- Final report, architecture diagrams, API documentation.
- Demo script and a backup demo video.

Done when: A clean-device demo works with no local setup.

7. Key Technical Decisions

Topic	Decision
Cold start	7-day features need history. Use a rule-based estimate for roughly the first 3-7 days, label it 'early estimate', then switch to the model.
Prediction target	Next-day stress score at or above 61 (Elevated or High). Store the actual outcome next day so real accuracy can be measured.
Risk thresholds	Low 0-30, Moderate 31-60, Elevated 61-80, High 81-100, stored in backend configuration, not in the UI.
Offline sync	Generate record UUIDs on the client so retries are idempotent; queue writes locally and flush when online.
Retraining	Collect consented real check-ins during usability testing; even 20-30 users makes the evaluation far more credible.
Chat safety order	Crisis detection first, then LLM, then output validation. The LLM never decides whether to show emergency help.
Data minimization	Send the LLM aggregated summaries only, never names, raw notes or identifiers unless the user has authorized it.
Explainability	Grouped factor impacts (Sleep, Mood, Activity, Stress Assessment, Heart Rate, Habits) from a logistic-regression surrogate.
Platform limits	HealthKit requires Mac, Apple developer account and a physical iPhone; make Android Health Connect the primary demo path.

8. Testing Strategy

Area	Approach	Target
Flutter	Unit tests for use cases, widget tests for key screens, integration test of the daily check-in flow	70%+ on domain layer
Node API	Jest + supertest per route; authorization matrix tests (user A cannot read user B)	80%+ on services
ML service	pytest for schema validation, feature builder parity with training, deterministic outputs	All edge cases covered
Security	OWASP API Top 10 review, rate-limit and token-expiry tests, dependency audit	No high-severity findings
Usability	5-10 participants complete a scripted task list; System Usability Scale questionnaire	SUS score 70+

9. Risks and Mitigations

Risk	Impact	Mitigation
Scope too large for timeline	High	Lock MVP first; treat wearables, dark mode and advanced analytics as stretch.
Synthetic data overstates accuracy	High	State the limitation in the report; validate with real consented check-ins.
Users disengage (sparse data)	Medium	Quick check-ins, reminders, graceful handling of missing values.
Sensitive health data exposure	High	Encryption, least privilege, audit logs, deletion tools, data minimization for the LLM.
Harmful chatbot response	High	Crisis pre-filter, output validation, clear non-clinician disclaimers, support screen always reachable.
Wearable API differences	Medium	Abstract behind one sync interface; develop on Android first.

10. Suggested Team Split (3 people)

Role	Owns	Phases
Person A - Mobile	Flutter app, offline queue, wearable integration	2, 6
Person B - Backend	Node API, database, security and privacy	1, 4, 7
Person C - ML and AI	ML service, chatbot, safety layer, testing and evaluation	3, 5, 8
Everyone	Setup and deployment/documentation	0, 9

11. Final Demo Checklist

- Register, complete profile setup, log mood and a stress assessment.
- Show tomorrow's risk with contributing factors and the 'not a diagnosis' wording.
- Show recommendations tied to the user's own data.
- Chat with the assistant, including one crisis-message demonstration.
- Export and delete data from the privacy center.
- Present the model comparison table and the synthetic-data limitation.